// T-OLIMPIANS · ARKUSZ REFERENCYJNY

Struktury danych w C++

Metody, złożoności czasowe i praktyczne wskazówki na konkursy informatyczne — OIJ, OI, IOI.

vector pair deque queue stack priority_queue set unordered_set map unordered_map bitset ordered_set (pbds)

01 vector — tablica dynamiczna

Używany jako: tablica, stos, bufor. Operacje na końcu są szybkie — na początku i w środku wolne.
W tabeli `all(v)` to skrót na `v.begin(), v.end()` (`#define all(v) v.begin(), v.end()`).

Operacja	Kod	Złożoność
Deklaracja z rozmiarem	`vector<int> v(n, 0);`	O(n)
Indeksowanie	`v[i]`	O(1)
Ustawienie elementu	`v[i] = x;`	O(1)
Dodanie na koniec	`v.push_back(x)`, `v.emplace_back(x)`	O(1)*
Usunięcie z końca	`v.pop_back()`	O(1)
Dodanie na początek/środek	`v.insert(v.begin() + i, x)`	O(n)
Usunięcie z początku/środka	`v.erase(v.begin() + i)`	O(n)
Pierwszy / ostatni	`v.front()`, `v.back()`	O(1)
Rozmiar / czy pusty	`v.size()`, `v.empty()`	O(1)
Zmiana rozmiaru	`v.resize(k)`	O(|k-n|)
Czyszczenie	`v.clear()`	O(n)
Sortowanie	`sort(all(v))`	O(n log n)
Odwrócenie	`reverse(all(v))`	O(n)
Wyszukanie wartości	`find(all(v), x)`	O(n)
Wyszukanie binarne (posortowany)	`binary_search(all(v), x)`	O(log n)
Pierwszy ≥ x / pierwszy > x	`lower_bound(all(v), x)`, `upper_bound(all(v), x)`	O(log n)
Suma elementów	`accumulate(all(v), 0LL)`	O(n)
Minimum / maksimum	`*min_element(all(v))`, `*max_element(all(v))`	O(n)
Usunięcie duplikatów (po sort)	`v.erase(unique(all(v)), v.end())`	O(n)

Dwa wymiary: `vector<vector<int>> dp(n, vector<int>(m, 0));` to tablica n×m wypełniona zerami.
Graf jako listy sąsiedztwa: `vector<vector<int>> g(n + 1);`, a potem `g[a].push_back(b);`.

* Amortyzowane — co jakiś czas push_back przepisuje całą tablicę, ale średnio wychodzi O(1).
Pułapka konkursowa: `v[i]` nie sprawdza zakresu — wyjście poza rozmiar to UB, zwykle zła odpowiedź zamiast błędu. Do funkcji przekazuj wektor jako `const vector<int>&` — bez `&` kopiujesz go w O(n).

02 pair / tuple — para i krotka

Sklejają kilka wartości w jedną. Porównywane leksykograficznie — dlatego świetnie sortują się w `vector` i działają jako klucz w `map` / `set`.

Operacja	Kod	Złożoność
Utworzenie pary	`pair<int, int> p = {a, b};` lub `make_pair(a, b)`	O(1)
Dostęp do pól	`p.first, p.second`	O(1)
Rozpakowanie (C++17)	`auto [a, b] = p;`	O(1)
Porównanie (leksykograficzne)	`p1 < p2`	O(1)
Wektor par	`vector<pair<int, int>> v;`	O(1)
Dodanie pary do wektora	`v.emplace_back(a, b);`	O(1)*
Sortowanie par	`sort(all(v))`	O(n log n)
Krotka 3+ wartości	`tuple<int, int, int> t = {a, b, c};`	O(1)
Dostęp w krotce	`get<0>(t), auto [a, b, c] = t;`	O(1)

Pułapka konkursowa: `sort` na `vector<pair>` porządkuje po `first`, a przy remisie po `second`. Potrzebujesz innej kolejności — wstaw do `first` to, po czym sortujesz (np. `-waga`, by uzyskać malejąco), albo podaj własny komparator.

03 deque — kolejka dwustronna

Szybka na obu końcach, a przy tym pozwala na indeksowanie w O(1) (inaczej niż `list` czy `deque` z Pythona). Baza kolejki monotonicznej.

Operacja	Kod	Złożoność
Dodanie na koniec	`dq.push_back(x)`	O(1)
Dodanie na początek	`dq.push_front(x)`	O(1)
Usunięcie z końca	`dq.pop_back()`	O(1)
Usunięcie z początku	`dq.pop_front()`	O(1)
Indeksowanie	`dq[i]`	O(1)
Pierwszy / ostatni	`dq.front(), dq.back()`	O(1)
Rozmiar / czy pusta	`dq.size(), dq.empty()`	O(1)
Wstawienie w środku	`dq.insert(dq.begin() + i, x)`	O(n)
Iteracja	`for (int x : dq)`	O(n)

Trik: minimum w oknie przesuwnym w O(n) — w `deque` trzymaj indeksy o rosnących wartościach: z przodu zdejmuj te, które wypadły z okna, z tyłu te, które są większe od nowego elementu.

04 queue / stack — kolejka i stos

Nakładki na deque. queue: pierwszy wszedł — pierwszy wychodzi (BFS). stack: ostatni wszedł — pierwszy wychodzi (DFS, nawiasy). Obie mają tylko po kilka metod.

Operacja	queue	stack	Złożoność
Deklaracja	`queue<int> q;`	`stack<int> st;`	O(1)
Dodanie	`q.push(x)`	`st.push(x)`	O(1)
Dodanie w miejscu	`q.emplace(x)`	`st.emplace(x)`	O(1)
Odczyt następnego	`q.front()`	`st.top()`	O(1)
Odczyt ostatniego	`q.back()`	—	O(1)
Usunięcie	`q.pop()`	`st.pop()`	O(1)
Rozmiar / czy pusta	`q.size()`, `q.empty()`	`st.size()`, `st.empty()`	O(1)

Pułapka konkursowa: pop() niczego nie zwraca — najpierw int x = q.front();, dopiero potem q.pop();. Odczyt z pustej struktury to UB, więc zawsze while (!q.empty()).

Do czego stos: sprawdzanie nawiasów, ONP, DFS bez rekurencji oraz najbliższy większy element (stos monotoniczny, O(n)).

05 priority_queue — kopiec

Domyślnie kopiec MAKSYMALNY — na górze największy element.

Operacja	Kod	Złożoność
Kopiec maksymalny	`priority_queue<int> pq;`	O(1)
Kopiec minimalny	`priority_queue<int, vector<int>, greater<int>> pq;`	O(1)
Wstawienie	`pq.push(x)`, `pq.emplace(x)`	O(log n)
Podgląd szczytu	`pq.top()`	O(1)
Usunięcie szczytu	`pq.pop()`	O(log n)
Rozmiar / czy pusta	`pq.size()`, `pq.empty()`	O(1)
Budowa z wektora	`priority_queue<int> pq(all(v));`	O(n)

Pułapka konkursowa: nie da się usunąć elementu ze środka ani przejrzeć kopca. W Dijkstrze trzymaj pary {dystans, wierzchołek} w kopcu minimalnym i pomijaj nieaktualne wpisy (if (d > dist[v]) continue;). Potrzebujesz usuwania dowolnego elementu → użyj set.

06 set / multiset — zbiór posortowany

Drzewo czerwono-czarne: elementy są zawsze posortowane. `set` trzyma unikalne, `multiset` dopuszcza duplikaty.

Operacja	Kod	Złożoność
Wstawienie	`s.insert(x)`	O(log n)
Usunięcie po wartości	`s.erase(x)`	O(log n)
Usunięcie po iteratorze	`s.erase(it)`	O(1) am.
Czy zawiera	`s.count(x)`, `s.contains(x)` (C++20)	O(log n)
Znalezienie elementu	`s.find(x)`	O(log n)
Najmniejszy	`*s.begin()`	O(1)
Największy	`*s.rbegin()`	O(1)
Pierwszy ≥ x	`s.lower_bound(x)`	O(log n)
Pierwszy > x	`s.upper_bound(x)`	O(log n)
Rozmiar / czy pusty	`s.size()`, `s.empty()`	O(1)
Iteracja (rosnąco)	`for (int x : s)`	O(n)

Pułapka konkursowa: zawsze `s.lower_bound(x)`, nigdy `lower_bound(all(s), x)` — ta druga wersja na secie działa w O(n). W multiset `s.erase(x)` kasuje WSZYSTKIE kopie; jedną usuniesz przez `s.erase(s.find(x))`.

07 unordered_set — zbiór haszujący

Bez kolejności, za to średnio O(1). Nie ma `lower_bound` / `upper_bound` ani iteracji po kolei.

Operacja	Kod	Średnia	Pesymist.
Wstawienie	`us.insert(x)`	O(1)	O(n)
Usunięcie	`us.erase(x)`	O(1)	O(n)
Czy zawiera	`us.count(x)`, `us.find(x)`	O(1)	O(n)
Rozmiar / czy pusty	`us.size()`, `us.empty()`	O(1)	—
Iteracja (losowa kolejność)	`for (int x : us)`	O(n)	—
Rezerwacja miejsca	`us.reserve(n)`	O(n)	—

Pułapka konkursowa: na OI trafiają się testy anty-hash, które psują unordered_set do O(n) na operację. Ratunek: zwykły set albo własny hash z losowym ziarnem (`chrono::steady_clock::now().time_since_epoch().count()`).

08 map / unordered_map — słownik

Przypisują kluczom wartości: `map<string, int> m;`. `map` trzyma klucze posortowane (drzewo), `unordered_map` wrzuca je do tablicy haszującej.

Operacja	Kod	map	unordered_map
Dostęp / wstawienie	`m[k]`, `m[k] = v;`	O(log n)	O(1) śr.
Czy jest klucz	`m.count(k)`, `m.find(k)`	O(log n)	O(1) śr.
Usunięcie	`m.erase(k)`	O(log n)	O(1) śr.
Wstawienie bez nadpisania	`m.emplace(k, v)`	O(log n)	O(1) śr.
Iteracja	`for (auto& [k, v] : m)`	O(n) rosnąco	O(n) losowo
Pierwszy klucz ≥ k	`m.lower_bound(k)`	O(log n)	brak
Najmniejszy klucz	`m.begin()→first`	O(1)	brak
Rozmiar / czy pusta	`m.size()`, `m.empty()`	O(1)	O(1)

Pułapka konkursowa: `m[k]` TWORZY brakujący klucz z wartością 0 — samo `if (m[k] > 0)` powiększa mapę i psuje `m.size()` oraz iterację. Do sprawdzania używaj `m.count(k)`.

09 bitset — tablica bitów

N bitów upakowanych po 64 w słowie — operacje na całym zbiorze są ok. 64× szybsze niż na `vector<bool>`. Rozmiar musi być stałą znaną w czasie kompilacji.

Operacja	Kod	Złożoność
Deklaracja	`bitset<1000006> b;`	O(n/64)
Ustawienie bitu	`b[i] = 1`, `b.set(i)`	O(1)
Wyzerowanie bitu / całości	`b.reset(i)`, `b.reset()`	O(1) / O(n/64)
Zanegowanie	`b.flip(i)`, `~b`	O(1) / O(n/64)
Odczyt bitu	`b[i]`, `b.test(i)`	O(1)
Liczba jedynek	`b.count()`	O(n/64)
Czy są jedynki	`b.any()`, `b.none()`, `b.all()`	O(n/64)
Suma / iloczyn / xor zbiorów	`a | b`, `a & b`, `a ^ b`	O(n/64)
Przesunięcie	`b << k`, `b >> k`	O(n/64)
Pierwsza / następna jedynka (GCC)	`b._Find_first()`, `b._Find_next(i)`	O(n/64)
Zamiana na liczbę / tekst	`b.to_ullong()`, `b.to_string()`	O(n/64)

Klasyka: sito Eratostenesa dla dużych N oraz plecak `dp |= dp << w;` — zamiast O(n·W) dostajesz O(n·W/64), czyli realnie ok. 64 razy szybciej.

10 ordered_set (pbds) — set z numerowaniem

Rozszerzenie GCC: `set`, który dodatkowo umie odpowiedzieć, ile elementów jest mniejszych od x oraz co stoi na k-tej pozycji. W wielu zadaniach zastępuje drzewo przedziałowe.

```
#include <ext/pb_ds/assoc_container.hpp>
#include <ext/pb_ds/tree_policy.hpp>
using namespace __gnu_pbds;

typedef tree<int, null_type, less<int>, rb_tree_tag,
             tree_order_statistics_node_update> ordered_set;

ordered_set A;
```

Operacja	Kod	Złożoność
Wstawienie	`A.insert(x)`	O(log n)
Usunięcie elementu równego x	`A.erase(x)`	O(log n)
Ile elementów mniejszych od x	`A.order_of_key(x)`	O(log n)
Element na k-tej pozycji	`*A.find_by_order(k)`	O(log n)
Pierwszy ≥ x / pierwszy > x	`A.lower_bound(x)`, `A.upper_bound(x)`	O(log n)
Rozmiar / czy pusty	`A.size()`, `A.empty()`	O(1)
Iteracja (rosnąco)	`for (int x : A)`	O(n)

Trzeci parametr szablonu decyduje o wszystkim:
1) `less<int>` / `greater<int>` → **ordered_set** — bez duplikatów, rosnąco / malejąco
2) `less_equal<int>` / `greater_equal<int>` → **ordered_multiset** — z duplikatami

Pułapka konkursowa: `order_of_key(x)` liczy elementy ostro MNIEJSZE od x, a `find_by_order(k)` numeruje od zera. W wersji z less_equal przestają działać find i erase(x) — usuwaj przez `A.erase(A.find_by_order(A.order_of_key(x)))` — a lower_bound i upper_bound zamieniają się rolami. Kompiluje się tylko pod g++.

11 ordered_set — typowe zastosowania

Zadanie	Rozwiązanie	Złożoność
k-ty najmniejszy element zbioru	`*A.find_by_order(k - 1)`	O(log n)
Pozycja x w posortowanym zbiorze	`A.order_of_key(x)`	O(log n)
Ile elementów w przedziale [a, b]	`A.order_of_key(b + 1) - A.order_of_key(a)`	O(log n)
Liczba inwersji w ciągu (różne wartości)	idź od końca: `A.order_of_key(v[i])`, potem `A.insert(v[i])`	O(n log n)

12 Tabela podsumowująca

Struktura	Dostęp	Wstawienie	Usunięcie	Posortowana
vector	O(1)	O(1) koniec	O(1) koniec	nie
deque	O(1)	O(1) oba końce	O(1) oba końce	nie
queue / stack	O(1) skraj	O(1)	O(1)	nie
priority_queue	O(1) szczyt	O(log n)	O(log n)	tylko szczyt
set / multiset	O(log n)	O(log n)	O(log n)	tak
map	O(log n)	O(log n)	O(log n)	tak (klucze)
unordered_set / map	O(1) śr.*	O(1) śr.*	O(1) śr.*	nie
bitset	O(1)	O(1)	O(1)	indeks = wartość
ordered_set	O(log n)	O(log n)	O(log n)	tak + k-ty

* Pesymistycznie O(n) na operację — przy testach anty-hash struktury haszujące potrafią się wywrócić.

13 Co wybrać do czego

Stos
stack — albo vector (push_back/pop_back)

Kolejka (BFS)
queue — push / front / pop

Kolejka priorytetowa / Dijkstra
priority_queue + greater<>

Zliczanie wystąpień
map<int, int> / unordered_map

Szybkie sprawdzenie „czy było”
unordered_set — małe wartości: vector<bool>

Zbiór trzymany w kolejności
set — z duplikatami: multiset

k-ty najmniejszy / ile mniejszych
ordered_set (pbds)

Minimum w oknie przesuwnym
deque — kolejka monotoniczna

Wielkie operacje na zbiorach bitów
bitset — sito, plecak

Klucz złożony z kilku wartości
pair / tuple

0-1 BFS
deque — push_front dla krawędzi 0

Tablica o rozmiarze z wejścia
vector<int> v(n) — zamiast tablic VLA

Na start zawodów: #include <bits/stdc++.h> wciąga całą bibliotekę standardową (g++), a using namespace std; oszczędza pisania std::. Do ordered_set dochodzą jeszcze dwa nagłówki pb_ds z sekcji 10.